

Graphs on real life

Contents

- 18.1. Commute Time Embedding
- 18.2. Exercises

Durante el curso hemos visto muchas bibliotecas y herramientas que pueden usarse para crear gráficos. Además, hemos visto cómo navegar por los datos y cómo crear un gráfico a partir de ellos. Por último, en las últimas semanas hemos visto cómo explorar el gráfico y cómo extraer información de él.

En esta sección, aplicaremos todo el conocimiento que hemos adquirido a un caso real. Utilizaremos grafos sintéticos para estudiar una técnica clásica de incrustación espectral: la **Inmersión en Tiempo de Conmutación**. No te preocupes si aún no estás familiarizado con todos los conceptos: te guiaremos durante el proceso y te proporcionaremos la información y el código necesarios para realizar las tareas.

Nota

Es importante mencionar que el objetivo de esta materia no es enseñarte cómo funcionan diferentes modelos de aprendizaje automático o deep learning, sino mostrarte cómo crear un gráfico a partir de un conjunto de datos y cómo explorarlo. Si te interesa saber más sobre los modelos, te animamos a visitar la documentación oficial de las bibliotecas o la documentación oficial de los modelos que te interesan.

18.1. Incorporación del tiempo de desplazamiento

En las clases anteriores, hemos visto cómo realizar caminatas aleatorias sobre un grafo y cómo usarlas para aprender representaciones de nodos (embeddings) con algoritmos como Node2Vec. En esta sección estudiaremos un enfoque diferente —y más clásico— de las incrustaciones de nodos basado en la **teoría espectral de grafos**: la **Inmersión en Tiempo de Conmutación**.

18.1.1. ¿Qué es el tiempo de desplazamiento?

Dado un grafo conexo no dirigido, el **tiempo de conmutación** entre dos nodos u y v es el número esperado de pasos que un caminante aleatorio necesita recorrer desde u y de vuelta a u . Formalmente, si denotamos por t_{uv} el tiempo de golpe (pasos esperados a alcanzar desde u), entonces el tiempo de desplazamiento es $C(u, v) = (t_{uv} + t_{vu})/2$.

[Saltar al contenido principal](#)

$$CT(u, v) = H(u, v) + H(v, u)$$

El tiempo de traslado tiene una expresión algebraica muy elegante. Si es el **pseudoinverso de Moore-Penrose** del grafo Laplaciano, entonces: $L^+ L$

$$CT(u, v) = \text{vol}(G) \cdot (L_{uu}^+ + L_{vv}^+ - 2L_{uv}^+)$$

donde es la suma de todos los grados de los nodos (es decir, el doble del número de aristas para grafos no dirigidos). $\text{vol}(G) = \sum_i d_i$

Esto significa que **el tiempo de conmutación entre dos nodos está codificado directamente en la pseudoinversa del laplaciano**. Los nodos fuertemente conectados (a través de muchos caminos cortos) tendrán un tiempo de desplazamiento corto; Los nodos que están débilmente conectados (distantes o conectados por pocos caminos) tendrán un tiempo de desplazamiento elevado.

18.1.2. ¿Por qué es útil el tiempo de desplazamiento?

- Es una medida **global**: a diferencia de la distancia de camino más corta, tiene en cuenta todos los caminos entre dos nodos, no solo el más corto.
- Está naturalmente relacionado con la estructura de **paseo aleatorio** del grafo.
- Da lugar a una **embebeding euclidiana** propiamente dicha: podemos incrustar nodos en tal que la distancia euclidiana al cuadrado entre dos nodos incrustados sea igual a su tiempo de conmutación (hasta una constante). $\mathbb{R}^k$

18.1.3. El Laplaciano de grafos

Antes de poder calcular la Inmersión en Tiempo de Conmutación, necesitamos entender el Laplaciano de Grafo. El laplaciano de un grafo se define como: G

$$L = D - A$$

donde es la **matriz de grados** (una matriz diagonal con , el grado del nodo) y es la matriz de **adyacencia**. $DD_{ii} = d_i$ iA

El Laplaciano posee varias propiedades importantes:

- Es **simétrica** y **semidefinida positiva**.
- Su valor propio más pequeño es siempre , con vector propio (el vector constante). **01**
- Para un grafo conexo, todos los demás valores propios son estrictamente positivos: .
 $0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_n$

Los valores propios y sus vectores propios correspondientes codifican la estructura global del grafo. El **valor de Fiedler** (segundo valor propio más pequeño) mide cuán bien

[Saltar al contenido principal](#)

conectado está el grafo — un pequeño indica que el grafo tiene un cuello de botella. $\lambda_i \phi_i \lambda_2$
 λ_2

18.1.4. Del Laplaciano al Embedding

La inmersión en tiempo de conmutación explota la autodescomposición del pseudoinverso laplaciano. El resultado clave es:

If we define the embedding of node as:

$$z_i = \left(\frac{1}{\sqrt{\lambda_2}} \phi_2(i), \frac{1}{\sqrt{\lambda_3}} \phi_3(i), \dots, \frac{1}{\sqrt{\lambda_k}} \phi_{k+1}(i) \right) \in \mathbb{R}^k$$

where $\phi_j(i)$ is the j -th component of the i -th eigenvector of (sorted by eigenvalue, skipping λ_1), then:

$$\|z_i - z_j\|^2 = \text{vol}(G) \cdot (L_{ii}^+ + L_{jj}^+ - 2L_{ij}^+) = CT(i, j)$$

That is: **the squared Euclidean distance between the embeddings equals the commute time between the nodes.**

Note

We skip the first eigenvector (associated with λ_1) because it is the constant vector and carries no structural information. The embedding is built from eigenvectors $\phi_2, \dots, \phi_{k+1}$, scaled by $\frac{1}{\sqrt{\lambda_j}}$.

18.1.5. Step-by-step Derivation

Let's trace through the full process to make things concrete.

Step 1 — Build the Laplacian.

Given the graph's adjacency matrix and degree matrix D , compute $L = D - A$.

Step 2 — Eigendecomposition.

Compute the eigenvalues and corresponding eigenvectors of L . $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$
 $\phi_1, \phi_2, \dots, \phi_n$

Step 3 — Select the top- k non-trivial eigenvectors.

Discard (constant vector). Select — the eigenvectors with the **smallest non-zero** eigenvalues. $\phi_2, \dots, \phi_{k+1}$

Step 4 — Scale each eigenvector.

Scale each selected eigenvector by $\frac{1}{\sqrt{\lambda_j}}$.

[Saltar al contenido principal](#)

$$\tilde{\phi}_j = \frac{1}{\sqrt{\lambda_j}} \phi_j$$

Step 5 — Stack into the embedding matrix.

The embedding matrix is formed by stacking the scaled eigenvectors as columns: $Z \in \mathbb{R}^{n \times k}$

$$Z = \left[\tilde{\phi}_2 \mid \tilde{\phi}_3 \mid \dots \mid \tilde{\phi}_{k+1} \right]$$

Row i of Z is the embedding of node i : $Z_{i,:}$

18.1.6. Code Snippet

Here is a code snippet that shows how to compute the Commute Time Embedding, where the function receives the graph and the number of dimensions: `G` `k`

```
import numpy as np
import networkx as nx
from scipy.linalg import eigh # eigh for symmetric matrices (more stable than eig)

def commute_time_embedding(G, k):
    """
    Compute the Commute Time Embedding of graph G in k dimensions.

    Parameters
    -----
    G : networkx.Graph
        An undirected connected graph.
    k : int
        Number of embedding dimensions (number of non-trivial eigenvectors to use).

    Returns
    -----
    Z : np.ndarray of shape (n_nodes, k)
        Embedding matrix. Row i is the embedding of node i.
        The squared Euclidean distance ||Z[i] - Z[j]||^2 equals the commute
        time CT(i, j).
    """
    # Step 1: Build the Laplacian
    L = nx.laplacian_matrix(G).toarray().astype(float)
    n = L.shape[0]

    # Step 2: Eigendecomposition – eigh returns eigenvalues in ascending order
    # We only need the first k+1 eigenpairs (0-indexed: 0 to k)
    eigenvalues, eigenvectors = eigh(L, subset_by_index=[0, k])
    # eigenvalues[j] = lambda_j, eigenvectors[:, j] = phi_j

    # Step 3: Skip lambda_0 = 0 (index 0); keep indices 1 to k
    # These correspond to lambda_2, ..., lambda_{k+1} in 1-indexed notation
    lam = eigenvalues[1:] # shape (k,)
    Phi = eigenvectors[:, 1:] # shape (n, k)

    # Safety check: all selected eigenvalues must be strictly positive
```

[Saltar al contenido principal](#)

```

        "Graph appears to be disconnected or k is too large. "
        "All non-trivial eigenvalues must be strictly positive."
    )

    # Step 4: Scale each eigenvector by 1/sqrt(lambda_j)
    # Broadcasting: divide each column j of Phi by sqrt(lam[j])
    Z = Phi / np.sqrt(lam)  # shape (n, k)

    return Z

```

Note

We use `scipy.linalg.eigh` instead of `numpy.linalg.eig` for two reasons: (1) it is specialized for **symmetric** matrices and is numerically more stable; (2) it returns eigenvalues in **ascending order**, which is exactly what we need to pick the smallest non-zero ones. The parameter `subset_by_index` tells SciPy to compute only the first eigenpairs, which is far more efficient than a full decomposition for large

graphs. `scipy.linalg.eigh` `numpy.linalg.eig` `eigh` `subset_by_index=[0, k]` `k+1`

18.1.7. Commute Time Embedding in Action

Now that we have the function, let's apply it to a concrete example. We will use the same **Stochastic Block Model (SBM)** graph from the previous classes — three communities of 20 nodes each, with high intra-community edge probability and low inter-community edge probability.

```

import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eigh

# --- Build the graph ---
G = nx.stochastic_block_model(
    [20, 20, 20],
    [
        [0.3, 0.01, 0.01],
        [0.01, 0.3, 0.01],
        [0.01, 0.01, 0.3]
    ],
    seed=42
)

# --- Visualize the original graph ---
plt.figure(figsize=(8, 8))
plt.title("SBM Graph - 3 communities")
pos = nx.spring_layout(G, seed=42)
colors = ['#e63946'] * 20 + ['#2a9d8f'] * 20 + ['#e9c46a'] * 20
nx.draw(G, pos, with_labels=True, node_color=colors, edge_color='a', node_size=100)
plt.show()

```

```
Z = commute_time_embedding(G, k=k)

print(f"Embedding matrix shape: {Z.shape}") # (60, 2)
print(f"Embedding of node 0: {Z[0]}")
print(f"Embedding of node 20: {Z[20]}")
print(f"Embedding of node 40: {Z[40]}")
```

Let's trace through what happens step by step for this graph:

Trace — Step 1: Build the Laplacian.

produces a 60×60 matrix. The diagonal entry (degree of node i); off-diagonal if $i \neq j$, else 0. `L = nx.laplacian_matrix(G).toarray()` $L_{ii} = d_i$ $L_{ij} = -1$ if $(i, j) \in E$ 0

Trace — Step 2: Eigendecomposition.

returns the 3 smallest eigenpairs: `eigh(L, subset_by_index=[0, 2])`

- $\lambda_1 \approx 0$ with (constant — discarded) $\phi_1 \approx \frac{1}{\sqrt{60}} \mathbf{1}$
- $\lambda_2 \approx 0.03$ with — first non-trivial eigenvector (separates one community from the rest) ϕ_2
- $\lambda_3 \approx 0.04$ with — second non-trivial eigenvector (separates the remaining two communities) ϕ_3

Trace — Step 3 & 4: Scale and stack.

```
lam = [0.03, 0.04]
Phi = [phi_2 | phi_3] # shape (60, 2)
Z = Phi / sqrt(lam) # each column divided by the square root of its eigenvalue
```

The resulting matrix has shape (60, 2). Row i is the 2D position of node i in the commute time space. `Z[i]`

Trace — Verify the commute time property.

Let's verify that the squared distance in the embedding approximates commute time:

```
# Compute commute time directly from the pseudoinverse of L
L_pinv = np.linalg.pinv(L)
vol_G = 2 * G.number_of_edges()

def commute_time_exact(i, j, L_pinv, vol_G):
    return vol_G * (L_pinv[i, i] + L_pinv[j, j] - 2 * L_pinv[i, j])

# Compare for a few pairs
pairs = [(0, 1), (0, 20), (0, 40)]
for i, j in pairs:
    ct_exact = commute_time_exact(i, j, L_pinv, vol_G)
    ct_embed = np.sum((Z[i] - Z[j])**2)
```

[Saltar al contenido principal](#)

You should observe that is a lower bound approximation of . The full commute time requires **all** non-trivial eigenvectors; using only of them gives an approximation. As , the approximation becomes exact. $\| \mathbf{ct_embed} - \mathbf{ct_exact} \|_F \rightarrow 0$

Note

The embedding with dimensions is an **approximation** of the true commute time, not an exact equality. The approximation improves as increases, but so does the computational cost. In practice, small values of (2–16) already capture the most important community structure of the graph, because the smallest eigenvalues carry the most global structural information. $k \ll n$

18.1.8. How to Visualize the Commute Time Embedding?

Since we chose , we can directly plot the embedding in 2D — no dimensionality reduction needed! This is one of the practical advantages of the Commute Time Embedding over random-walk-based methods: **we directly control the dimensionality** of the output. $k=2$

```
# --- Visualize the 2D embedding ---
community_colors = ['#e63946'] * 20 + ['#2a9d8f'] * 20 + ['#e9c46a'] * 20

plt.figure(figsize=(8, 8))
plt.scatter(Z[:, 0], Z[:, 1], c=community_colors, s=80, edgecolors='black', linewidth=1)
plt.title("Commute Time Embedding (k=2)")
plt.xlabel("Dimension 1 ($\phi_2 / \sqrt{\lambda_2}$)")
plt.ylabel("Dimension 2 ($\phi_3 / \sqrt{\lambda_3}$)")
plt.grid(True, alpha=0.3)
plt.show()
```

If the community structure is strong, the three communities should appear as three separate clusters in the 2D plot. This is because nodes within the same community have short commute times (small squared distance in the embedding), while nodes in different communities have long commute times (large squared distance).

For higher-dimensional embeddings (), we can use PCA or t-SNE to project down to 2D for visualization: $k > 2$

```
from sklearn.decomposition import PCA

# Compute a richer embedding with more dimensions
k_high = 16
Z_high = commute_time_embedding(G, k=k_high)
print(f"High-dimensional embedding shape: {Z_high.shape}") # (60, 16)

# Project to 2D with PCA
pca = PCA(n_components=2)
Z_pca = pca.fit_transform(Z_high)
```

[Saltar al contenido principal](#)

```
plt.scatter(Z_pca[:, 0], Z_pca[:, 1], c=community_colors, s=80, edgecolors='black',
plt.title(f"Commute Time Embedding (k={k_high}) – projected with PCA")
plt.xlabel("PCA component 1")
plt.ylabel("PCA component 2")
plt.grid(True, alpha=0.3)
plt.show()
```

18.2. Exercises

18.2.1. Exercise 1

As we did in the previous classes, after reading the previous section, you should be able to create your Jupyter Notebook and write a report about all the things you have learned in this section. In order to have a good structure, we recommend you to follow exactly the same structure as in this notebook. **IMPORTANT: I want a very detailed report, with all the**

previous practices where we made a trace of the algorithms.

18.2.2. Exercise 2

Now that you have learned how to use the Commute Time Embedding to learn the embeddings of the nodes in a graph, apply it to the graph that we have generated in the previous classes (4 + 2 Graphs). Then, visualize the embeddings of the nodes in a low-dimensional space using PCA or t-SNE (when) or directly (when). Answer the following questions: $k > 2$ $k = 2$

- What can you say about the embeddings of the nodes in the graph?
- Are the nodes that are close in the graph also close in the embedding space?
- What can you say about the quality of the embeddings?
- How does changing (the number of dimensions) affect the quality of the embedding? k
- If you could improve the quality of the embeddings, how would you do it?

Note

The aim of this exercise is to show you how to apply the Commute Time Embedding to a graph and how to visualize the node embeddings in a low-dimensional space. But also, to play with the parameter of the embedding function and to see how it affects the quality of the representation. k